

```
#include "BluetoothSerial.h"

#include "soc/soc.h"

#include "soc/rtc_cntl_reg.h"
```

```
BluetoothSerial SerialBT;
```

```
// --- HARDWARE PIN DEFINITIONS ---
```

```
// TB6612FNG Motor Pins
```

```
const int PWMA = 22;
```

```
const int AIN1 = 21;
```

```
const int AIN2 = 19;
```

```
const int PWMB = 23;
```

```
const int BIN1 = 18;
```

```
const int BIN2 = 5;
```

```
// HC-SR04 Ultrasonic Pins
```

```
const int TRIG_PIN = 12;
```

```
const int ECHO_PIN = 14;
```

```
String currentState = "STOPPED";
```

```
float currentDistance = 100.0;
```

```
char lastCommand = 'X';
```

```
void setup() {
```

```
    // 1. DISABLE BROWNOUT DETECTOR (Prevents crashes on battery power)
```

```
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
```

```
    // 2. INITIALIZE PINS
```

```
pinMode(PWMA, OUTPUT); pinMode(AIN1, OUTPUT); pinMode(AIN2, OUTPUT);  
pinMode(PWMB, OUTPUT); pinMode(BIN1, OUTPUT); pinMode(BIN2, OUTPUT);  
pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
```

```
// 3. START COMMUNICATIONS
```

```
Serial.begin(115200);
```

```
SerialBT.begin("ADAS_Dumper");
```

```
}
```

```
// Motor Control Helper Functions
```

```
void stopMotors() {
```

```
    analogWrite(PWMA, 0); analogWrite(PWMB, 0);
```

```
    digitalWrite(AIN1, LOW); digitalWrite(AIN2, LOW);
```

```
    digitalWrite(BIN1, LOW); digitalWrite(BIN2, LOW);
```

```
}
```

```
void driveForward() {
```

```
    digitalWrite(AIN1, HIGH); digitalWrite(AIN2, LOW);
```

```
    digitalWrite(BIN1, HIGH); digitalWrite(BIN2, LOW);
```

```
    analogWrite(PWMA, 150); analogWrite(PWMB, 150); // Adjust speed 0-255
```

```
}
```

```
void reverse() {
```

```
    digitalWrite(AIN1, LOW); digitalWrite(AIN2, HIGH);
```

```
    digitalWrite(BIN1, LOW); digitalWrite(BIN2, HIGH);
```

```
    analogWrite(PWMA, 150); analogWrite(PWMB, 150);
```

```
}
```

```
void turnLeft() {
```

```
    digitalWrite(AIN1, LOW); digitalWrite(AIN2, HIGH);
```

```
    digitalWrite(BIN1, HIGH); digitalWrite(BIN2, LOW);
```

```

    analogWrite(PWMA, 130); analogWrite(PWMB, 130);
}

void turnRight() {
    digitalWrite(AIN1, HIGH); digitalWrite(AIN2, LOW);
    digitalWrite(BIN1, LOW); digitalWrite(BIN2, HIGH);
    analogWrite(PWMA, 130); analogWrite(PWMB, 130);
}

void loop() {
    // 1. READ REAL SENSOR PING
    digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30ms timeout

    if (duration == 0) {
        currentDistance = 100.0; // Out of range
    } else {
        currentDistance = (duration * 0.0343) / 2;
    }

    // 2. READ BLUETOOTH COMMANDS FROM PYTHON
    if (SerialBT.available() > 0) {
        lastCommand = SerialBT.read();
    }

    // 3. SIMULATION OVERRIDE (For the presentation demo)
    if (lastCommand == 'O') {

```

```
    currentDistance = 15.0; // Force an obstacle drop
}
else if (lastCommand == 'A' || lastCommand == 'D' || lastCommand == 'S') {
    if (currentDistance == 15.0) currentDistance = 100.0; // Reset after dodge
}
```

// 4. HARDWARE FAILSAFE LOGIC

```
if (currentDistance <= 20.0 && lastCommand == 'W') {
    currentState = "EMERGENCY_BRAKE";
    stopMotors(); // ADAS Overrides Operator
}
else {
    if (lastCommand == 'W') { currentState = "MOVING"; driveForward(); }
    else if (lastCommand == 'A') { currentState = "TURNING"; turnLeft(); }
    else if (lastCommand == 'D') { currentState = "TURNING"; turnRight(); }
    else if (lastCommand == 'S') { currentState = "REVERSING"; reverse(); }
    else { currentState = "STOPPED"; stopMotors(); }
}
```

// 5. TRANSMIT REAL TELEMETRY OVER BLUETOOTH

```
SerialBT.print(currentDistance);

SerialBT.print("");

SerialBT.println(currentState);

delay(50); // 20Hz Update Rate
}
```